

Castelnuovo di Porto — nonlinear time-history

Step-by-step instructions for running the analysis on the desktop machine (Intel i9-9900KF, 8 cores, 32 GB RAM). Everything runs inside a Docker container, so nothing needs to be installed on Windows except Docker Desktop and Git.

0. What you are about to run, and what it costs

A nonlinear dynamic (time-history) analysis of a masonry building aggregate: about 35,000 nodes and 120,000 tetrahedral elements, with a damage-plasticity material, contact interfaces between units, and multi-point constraints that restore wall continuity the source geometry had lost.

Quantity	Value	Where it comes from
Seismic input	1979 Montenegro, Ulcinj–Hotel Albatros, longitudinal component as delivered in SERA-AIMS Run 2.1 — PGA 0.593 g at $t = 12.345$ s	Same record as Chapter 6; committed in the repository already converted to m/s^2
Time step / duration	0.005 s, 19.5 s $\rightarrow$ 3900 increments	Chapter 6's own setup for this record
Integrator	TRBDF2	Chapter 6 — composite scheme, damps the spurious high-frequency oscillations that contact and softening produce
Solver / iteration	UmfPack, Newton with line search, energy-increment test 1×10^{-3} , 200 iterations	Chapter 6
Damping	Rayleigh, 3% at modes 1 and 10 (5.786 and 10.695 Hz)	Chapter 6's convention; frequencies from the 80-mode modal run
Expected wall-clock time	roughly 15–40 hours	Extrapolated from a measured ~ 5 s per stiffness factorisation on a 6-core i7-10750H; see the note below

About the run time. The estimate is a projection, not a measurement on this machine. It was obtained by timing one stiffness factorisation on the real model (≈ 5 s at 101,634 free degrees of freedom) and multiplying by steps $\times$ iterations. The spread is wide because the number of Newton iterations per step depends on how much the masonry cracks, which is not known in advance.

More cores will not make this faster. The solver used here (UmfPack) is sequential — it runs on one core. The i9-9900KF helps through its higher single-core clock, not through having 8 cores. Using all cores would require the parallel route (OpenSeesMP + MUMPS), which is not what this script does and carries an unresolved risk with the multi-point constraints.

1. Install the prerequisites

1. **Docker Desktop** — <https://www.docker.com/products/docker-desktop/>

During installation accept the WSL 2 backend. Reboot when asked, then start Docker Desktop and wait until the whale icon stops animating.

2. **Git** — <https://git-scm.com/download/win> (accept all defaults).

1a. Give Docker enough memory (important)

On Windows, Docker Desktop runs inside WSL 2, which by default caps itself at half the host RAM and may release it poorly. A direct sparse factorisation of this model is memory-hungry. Create the file `C:\Users\<your-user>\.wslconfig` with this content, then run `wsl --shutdown` and restart Docker Desktop:

```
[wsl2]
memory=24GB
processors=8
swap=8GB
```

24 GB of the 32 GB leaves Windows enough to stay responsive. If the analysis is later killed without an error message, that is the classic symptom of running out of memory — raise this value and retry.

2. Get the code

```
cd C:\
mkdir work
cd C:\work
git clone https://github.com/mlaura1996/modularBIMtoFEM.git
cd modularBIMtoFEM
git checkout chapter7-castelnuovo
git log --oneline -3
```

The last command should list recent commits mentioning the time-history and the junction ties. The branch is `chapter7-castelnuovo` — not `main`.

If the branch is missing or out of date: the commits may not have been pushed yet. Ask Maria Laura to run `git push origin chapter7-castelnuovo` on her laptop first, then `git fetch && git checkout chapter7-castelnuovo` here.

3. Get the Docker image

The image contains OpenSees 3.8.0 compiled with MUMPS, gms, and the whole Python environment — about 7 GB. There are two ways to get it; **the first is strongly preferred**.

Option A — copy the exact image (preferred)

On Maria Laura's laptop:

```
docker save modularbimtofem-opensees-mp:dev | gzip > castelnuovo-image.tar.gz
```

Copy that file across (external drive or network share), then here:

```
docker load -i D:\castelnuovo-image.tar.gz
docker images modularbimtofem-opensees-mp
```

This is preferred because it gives bit-identical binaries. Results computed here are then comparable with everything already in the thesis.

Option B — rebuild it

```
cd C:\work\modularBIMtoFEM
docker build -f docker/opensees/Dockerfile -t modularbimtofem-opensees-mp:dev .
```

Takes 20–40 minutes and needs internet. The build pins the versions that matter (MUMPS commit, OpenSees v3.8.0, Python 3.10.12, numpy, pandas, matplotlib, ifcopenshell), but the Ubuntu system packages are not pinned, so a rebuild months later is not guaranteed to be identical. Use Option A unless it is impossible.

4. Verify the machine reproduces the model

Do not skip this step. It takes about 20 minutes and it is the difference between a 30-hour run that is usable and a 30-hour run that has to be thrown away. It re-runs the whole pipeline and compares node counts, total volume, junction count, constraint count, self-weight, base reaction and all ten eigenvalues against reference values committed in the repository.

Open **PowerShell** in `C:\work\modularBIMtoFEM` and run:

```
docker run --rm -v "${PWD}:/app" -w /app `
  --entrypoint conda modularbimtofem-opensees-mp:dev `
  run --no-capture-output -n appenv `
  python docker/opensees/verify_environment.py
```

The last line must read:

```
PASS - this machine reproduces the reference model.
```

If it says...	What it means	What to do
FAIL on <code>mesh_nodes</code> or <code>mesh_elements</code>	gmsh or the OCC geometry kernel behaved differently, so the model was meshed differently	You are not using the same image — use Option A
FAIL on <code>total_volume_m3</code>	The input STEP geometry differs	The repository checkout is wrong or incomplete; re-clone
FAIL on <code>equaldof_constraints</code>	The junction ties came out different — the walls are connected differently than in the reference model	Stop and report this; do not run the analysis
FAIL on eigenvalues only, by $\sim 1e-5$ or more	Same model, different solver behaviour	Report the numbers before continuing
Killed with no message	Out of memory	Raise <code>memory</code> in <code>.wslconfig</code> (step 1a)

5. Smoke test — 10 minutes, and it checks two specific things

```
docker run --rm -v "${PWD}:/app" -w /app `
  --entrypoint conda modularbimtofem-opensees-mp:dev `
  run --no-capture-output -n appenv `
  python docker/opensees/timehistory_castelnuovo.py --smoke
```

This uses a coarse mesh and a handful of time steps. It is not a physical result — it exists to check the wiring. Two lines in its output matter more than anything else:

5a. The conflict guard

Look for either:

```
conflict guard passed: no tied node is also an interface duplicate
```

or:

```
ABORT: N node(s) are both tied and duplicated at a contact interface
```

If it aborts, stop and report it. This is a known, anticipated conflict, not a bug to work around. Two walls (volumes 91 and 107) carry both a contact interface and junction ties. If a node belongs to both, the constraint ends up pointing at a node its own wall no longer uses: OpenSees accepts it, nothing errors, the constraint carries no load, and the wall junction is silently open again — which is the exact defect this model was built to fix. Resolving it is a modelling decision (which face should transmit contact and which continuity), so it is deliberately not automated.

5b. The recorder check

Look for the block beginning `--- recorder check after 5 steps ---` and note which recorders report `EMPTY`.

The damage-output response names for this material (`ASDConcrete3D`) could not be verified in advance, because the analysis was not run on the laptop. OpenSees does *not* raise an error for an unknown element response — it creates the recorder and writes only the time column, with no data. The check counts data columns rather than file size for exactly that reason. So the script registers several candidate names and reports which ones actually produced data. **Send that list back before starting the full run.** The strain and stress recorders, the displacements and the reactions are standard and are expected to work; it is specifically the `solid_damage*` ones that need confirming.

6. The full run

Run it **detached**, so closing the window does not kill it:

```
docker run -d --name castelnuovo-th `
-v "${PWD}:/app" -w /app `
--memory=22g `
--entrypoint conda modularbimto fem-opensees-mp:dev `
run --no-capture-output -n appenv `
python docker/opensees/timehistory_castelnuovo.py
```

Follow the progress:

```
docker logs -f castelnuovo-th
```

or read the log file directly, which is also written as the run proceeds:

```
Get-Content -Wait output\castelnuovo\recorders_timehistory\progress.log
```

Every 2% of the run it prints a line like:

```
[14:32:07] step 780/3900 t=3.900 s elapsed 3.41 h ETA 13.65 h
```

The first such line gives a real estimate for this machine. The stages before the transient starts take their own time and are normal:

Stage	Roughly
Geometry, two-pass meshing	10–15 min
Building elements (one damage material per element)	20–60 min — the slowest setup stage, it is not stuck
Static self-weight (10 nonlinear steps)	5–15 min
Transient, 3900 steps	the rest

Protect the run

- **Disable automatic restarts:** Settings → Windows Update → Advanced options → set active hours, and turn off "Restart this device as soon as possible".
- **Disable sleep and hibernation:** Settings → System → Power → Screen and sleep → set both to *Never*. Sleeping the host suspends the container.
- **Leave Docker Desktop running.** Do not sign out of Windows.

There is no restart capability. The damage material keeps its state inside each element; that state cannot be rebuilt from the recorded displacements, and OpenSees' own save/restore does not cover it reliably. If the run dies at hour 20 it starts again from zero. This is why the three points above matter. The recorders are written continuously, so an interruption at least leaves the response up to that moment usable.

7. Post-processing

```
docker run --rm -v "${PWD}:/app" -w /app `
  --entrypoint conda modularbimtofem-opensees-mp:dev `
  run --no-capture-output -n appenv `
  python scripts/postprocess_timehistory.py `
  output/castelnuovo/recorders_timehistory
```

This writes, into the same folder:

File	Contents
<code>principal_strains.csv</code>	Per element: peak tensile and peak compressive principal strain and when each occurred. Computed here, not by OpenSees — principal strains are the eigenvalues of the strain tensor, and OpenSees records only the six components.
<code>damage_map.csv</code>	Per element: peak damage. The data behind a damage-pattern figure.
<code>response.csv</code>	Per step: roof displacement Rd, base shear BS, base shear coefficient BS/W, global drift ratio. The hysteretic loops are BS/W against the drift ratio.
<code>energy.csv</code>	Per step: cumulative dissipated hysteretic energy.
<code>interface_opening.csv</code>	Per step: maximum opening across the unit-to-unit contact interfaces (ld).
<code>summary_postprocess.json</code>	All the peaks and totals in one place.

8. What to send back

The result files are small compared with the raw recorders. Zip and return:

```
Compress-Archive -Path output\castelnuovo\recorders_timehistory\*.csv,`
  output\castelnuovo\recorders_timehistory\*.json,`
  output\castelnuovo\recorders_timehistory\progress.log `
  -DestinationPath castelnuovo-results.zip
```

Include `progress.log` — it records the wall-clock time, any steps that needed subdivision, and the recorder check, all of which belong in the thesis record.

9. Troubleshooting

Symptom	Cause	Fix
<code>docker: command not found</code>	Docker Desktop not running or not installed	Start Docker Desktop, wait for the icon to settle
<code>invalid reference format</code> or path errors	PowerShell vs cmd quoting	Use PowerShell, keep the quotes around <code>"\${PWD}:/app"</code> , and use the backtick <code>`</code> for line continuation (not <code>\</code>)
Container exits immediately, no output	Out of memory	Raise <code>.wslconfig</code> memory; lower <code>--memory</code> only if Windows itself becomes unusable
<code>ABORT: gravity did not converge</code>	The model cannot carry its own weight — a modelling problem, not a machine problem	Stop and report. A time-history on such a model is meaningless.
Many <code>step N</code> needed subdivision lines	Normal with contact and cracking; the step is retried at one tenth	Nothing — but it slows the run, so the ETA will grow
<code>ABORT: 5 steps failed to converge even subdivided</code>	The analysis genuinely cannot proceed	Stop. The response up to that point is recorded and usable, but it must be reported as a run that stopped early, not as a complete one.
Run is much slower than the ETA suggested	Thermal throttling, or another heavy program is competing	Close other work; check the CPU is not being shared

10. Quick reference

Purpose	Command
Verify the machine	<code>python docker/opensees/verify_environment.py</code>
Smoke test	<code>python docker/opensees/timehistory_castelnuovo.py --smoke</code>
Full run	<code>python docker/opensees/timehistory_castelnuovo.py</code>
Excite in Y instead of X	<code>... timehistory_castelnuovo.py --dof 2</code>
Shorter window (e.g. 5 s, for testing)	<code>... timehistory_castelnuovo.py --duration 5</code>
Post-process	<code>python scripts/postprocess_timehistory.py <run_dir></code>
Stop a detached run	<code>docker stop castelnuovo-th</code>
Remove a finished container	<code>docker rm castelnuovo-th</code>

All the Python commands above are prefixed by the same Docker wrapper shown in step 4. Nothing is ever run with the Windows Python directly.